ML Factory

A.SCHUTZ

Description : `MLflow` est la plateforme centrale pour gérer le cycle de vie du ML. Le serveur de tracking centralise les paramètres et les métriques de vos expériences dans une base de données (SQLite par défaut).

Il expose une interface web (`port 5000`) et une API REST que vos scripts Python interrogent pour enregistrer chaque tentative d'entraînement.

```yaml
mlflow:
  image: ghcr.io/mlflow/mlflow
  container_name: mlflow_server
  depends_on: [minio]
  ports: ["5000:5000"]
  environment:
    - MLFLOW_S3_ENDPOINT_URL=http://minio:9000
    - AWS_ACCESS_KEY_ID=minioadmin
    - AWS_SECRET_ACCESS_KEY=minioadmin
  command: >
    mlflow server
    --backend-store-uri sqlite:///mlflow.db
    --default-artifact-root s3://mlflow/
    --host 0.0.0.0
```

Modèle Registry



Le Model Registry est le catalogue officiel de vos modèles. Comme `MLflow` ne stocke pas de gros fichiers, on utilise `MinIO`, un stockage objet compatible S3, pour conserver les fichiers binaires (`.pkl`, `.onnx`).

```yaml
services:
  minio:
    image: minio/minio
    container_name: minio
    ports:
      - "9000:9000" # API S3
      - "9001:9001" # Interface Web (Console)
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    command: server /data --console-address ":9001"
    volumes:
      - minio_data:/data  # dossier /data de MinIO

volumes:
  minio_data:
```

https://github.com/minio/minio

MinIO

Modèle Registry



Création du bucket

Boto3 assure la communication entre mlflow et mnio

```python
import mlflow.sklearn
from mlflow.tracking import MlflowClient
import boto3

os.environ['MLFLOW_S3_ENDPOINT_URL'] = "http://localhost:9000"
os.environ['AWS_ACCESS_KEY_ID'] = "minioadmin"
os.environ['AWS_SECRET_ACCESS_KEY'] = "minioadmin"

def prepare_minio():
    """Vérifie si le bucket 'mlflow' existe, sinon le crée"""

    s3 = boto3.client('s3', endpoint_url=os.environ['MLFLOW_S3_ENDPOINT_URL'])

    buckets = [b['Name'] for b in s3.list_buckets()['Buckets']]

    if 'mlflow' not in buckets:
        s3.create_bucket(Bucket='mlflow')
        print("Bucket 'mlflow' créé avec succès.")
```

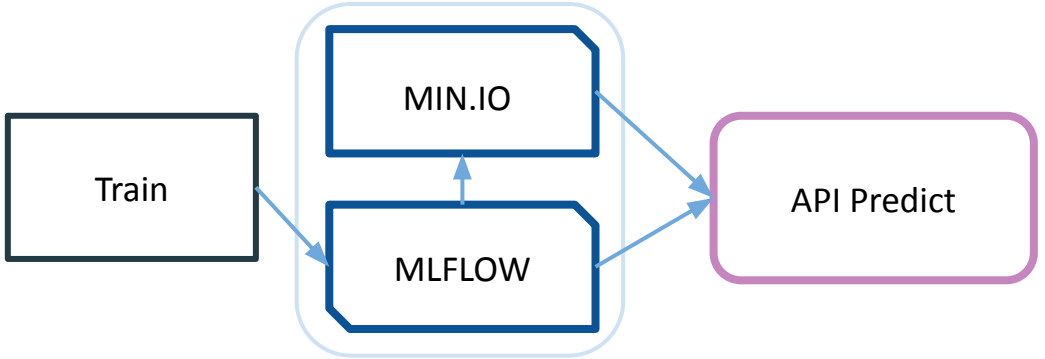
```python
def train_and_register():
    # Configuration du serveur de tracking
    mlflow.set_tracking_uri("http://localhost:5000")
    mlflow.set_experiment("My_Experiment")
    # Chargement des données train/test

    with mlflow.start_run() as run:
        # Entraînement
        # Log des paramètres et metrics
        mlflow.log_params(params)
        accuracy = model.score(X_test, y_test)
        mlflow.log_metric("accuracy", accuracy)
        # 2. Enregistrement dans MinIO ET dans le Model Registry
        # On définit le nom du modèle dans le catalogue
        model_name = "model_name"
        mlflow.sklearn.log_model(
            sk_model=model,
            artifact_path="model",
            registered_model_name=model_name
        )
    # 3. Gestion de l'Alias 'Production' via MlflowClient
    client = MlflowClient()

    # On récupère la toute dernière version créée
    latest_version = client.get_latest_versions(model_name, stages=["None"])[0].version

    # On lui attribue l'alias 'Production'
    client.set_registered_model_alias(model_name, "Production", latest_version)
```

MLflow Client



Le MlflowClient est l'outil d'administration pour gérer les modèles. Une fois l'entraînement terminé, on enregistre le modèle sous un nom unique dans le registre.

On utilise ensuite des Alias (comme Production ou Staging) pour identifier la version active. Cela permet de séparer le numéro de version technique de son usage métier.

```
from mlflow import MlflowClient
client = MlflowClient()

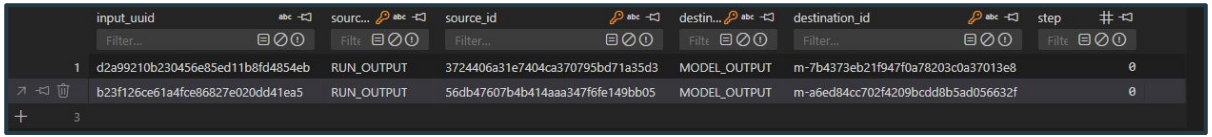
# Enregistre la version 1 du modèle 'iris' sous l'alias 'Production'
client.set_registered_model_alias("iris_model", "Production", "1")
```

Inférence Dynamique avec FastAPI

Côté API, on ne charge plus un fichier local `model.pkl`. FastAPI interroge le `Registry` via le `client MLflow` pour télécharger la version marquée en Production.

L'API vérifie à chaque exécution si l'alias a changé. Si vous passez la V2 en production dans MLflow, l'API la récupère automatiquement sans redémarrer.

mlflow.db:inputs



	input_uuid	sourc...	source_id	destin...	destination_id	step
	Filter...	Filte	Filter...	Filte	Filter...	Filte
1	d2a99210b230456e85ed11b8fd4854eb	RUN_OUTPUT	3724406a31e7404ca370795bd71a35d3	MODEL_OUTPUT	m-7b4373eb21f947f0a78203c0a37013e8	0
	b23f126ce61a4fce86827e020dd41ea5	RUN_OUTPUT	56db47607b4b414aaa347f6fe149bb05	MODEL_OUTPUT	m-a6ed84cc702f4209bcdd8b5ad056632f	0
3						

```python
# Initialisation du client MLflow
client = MlflowClient(tracking_uri=MLFLOW_URI)

# Cache pour éviter de recharger le modèle si la version n'a pas changé
state = {
    "model": None,
    "version": None
}

def load_production_model():
    """Vérifie la version en production et recharge si nécessaire."""
    try:
        # On demande au Registry quelle est la version actuelle de l'alias 'Production'
        alias_info = client.get_model_version_by_alias(MODEL_NAME, MODEL_ALIAS)
        prod_version = alias_info.version

        # Si le modèle n'est pas en cache ou si la version a changé sur MLflow
        if state["model"] is None or prod_version != state["version"]:
            print(f"🔄 Chargement de la version {prod_version} depuis MinIO...")
            model_uri = f"models:/{MODEL_NAME}@{MODEL_ALIAS}"
            state["model"] = mlflow.pyfunc.load_model(model_uri)
            state["version"] = prod_version

        return state["model"]
    except Exception as e:
        raise HTTPException(status_code=404, detail=f"Erreur MLflow: {str(e)}")
```